Casualty and Disaster Insurance Management System

Comprehensive Technical Documentation

---

1. Project Overview

Purpose of the System

The **Casualty and Disaster Insurance Management System** is a robust, full-stack application designed to streamline the lifecycle of insurance policies and claims. It provides an integrated platform for customers to browse insurance plans, request customized policies, complete purchases, and raise claims. It also empowers administrative staff, agents, and claims officers to seamlessly manage users, reviews, approvals, and payouts in an automated and transparent manner.

Problem the System Solves

Traditional insurance management often involves fragmented systems, manual paperwork, and delays in policy approvals and claim settlements. This system solves these issues by:

- Digitizing the entire workflow from plan discovery to claim settlement.
- Automating document tracking (PAN and Address proofs) and risk scoring.
- Ensuring role-based access control, so each stakeholder (Admin, Agent, Customer, Claims Officer) has a dedicated interface and workflow.
- Generating automated financial documents (Invoices, Receipts) securely stored and easily retrievable.

High-Level System Architecture

The application employs a modern **Client-Server Architecture**:

- **Client**: A Single Page Application (SPA) built with Angular that communicates with the API via HTTP requests.
- **Server**: A RESTful API built with ASP.NET Core 8 following **Clean Architecture** principles.
- **Database**: SQL Server accessed via Entity Framework Core (EF Core) as the ORM.
- **Third-Party Services**: Vercel Blob for cloud file storage (receipts, documents) and QuestPDF for server-side PDF generation.

Technology Stack Used

- **Backend:** .NET 8, C#, ASP.NET Core Web API, Entity Framework Core, SQL Server.
- **Architecture:** Clean Architecture (Domain, Application, Infrastructure, API layers).
- **Frontend:** Angular 19, TypeScript, HTML, CSS, RxJS, Angular Router.
- **Third-Party Libraries / Integrations:** QuestPDF (PDF generation), Vercel Blob Storage, BCrypt (Password Hashing), Microsoft.AspNetCore.Authentication.JwtBearer.

---

2. System Architecture

The project is highly structured to ensure scalability, maintainability, and separation of concerns.

Backend (Clean Architecture)

Clean Architecture organizes the code into concentric layers where dependencies always point inwards toward the Domain.

1. **Domain Layer (`Domain`)**
    - Contains core business logic and entities.
    - Defines Models (e.g., `User` , `Policy` , `Claim` , `Plan`), Enums (`UserRole` , `ClaimStatus` , `PaymentStatus`).
    - *No external dependencies.*
2. **Application Layer (`Application`)**
    - Handles the use cases and business rules orchestrating the entities.
    - Contains Data Transfer Objects (DTOs), Interfaces (`IAuthService` , `IVercelBlobService` , `IUserRepository`), and core feature Services (`ClaimService` , `PolicyService`).
    - Uses the Domain layer but is oblivious to the actual database or web API.
3. **Infrastructure Layer (`Infrastructure`)**
    - Implements the interfaces defined in the Application layer.
    - Contains the `AppDbContext` (EF Core database context), Repository implementations, and third-party service concrete classes (`VercelBlobService` , `InvoiceGeneratorService`).
    - Handles external resources like File Storage, Email/Notification providers (if any), and Database.
4. **API Layer (`API`)**
    - The entry point of the backend. Contains ASP.NET Core Controllers (`ClaimsController` , `PoliciesController`).
    - Routes incoming HTTP requests to the Application services and formats responses.
    - Handles JWT Authentication and Role-Based Authorization.

Frontend (Angular Architecture)

The frontend is built using Angular standalone components architecture.

- **Components:** Modular pieces of the UI grouped by role (`admin` , `agent` , `customer` , `claims-officer`) and `common` logic.
- **Services:** Singleton classes used to encapsulate HTTP calls to the backend APIs. Examples include `ClaimService` , `PolicyRequestService` , `AuthService` .
- **Routing:** Handled in `app.routes.ts` . Routes are protected using Angular Route Guards (`authGuard` , `roleGuard` , `guestGuard`).
- **State Handling:** Usually managed locally in component classes or via RxJS observables within services for cross-component communication.

Communication: The Angular frontend communicates with the .NET Backend via standard HTTP/HTTPS protocols using the `HttpClient` . All requests targeting secure endpoints include a Bearer JWT Token in the Authorization header.

---

3. User Roles and Responsibilities

The system relies on a Role-Based Access Control (RBAC) model with **4 distinct roles**:

1. Admin

- **Responsibilities:** Overall system management.
- **Actions:**
    - Manage all users (view, delete, update roles).
    - Register new company employees (Agents and Claims Officers).

    - Add, update, and deactivate Insurance Plans.
    - View all metrics, policies, and claims globally.

2. Agent

- **Responsibilities:** Facilitates policy creation for customers.
- **Actions:**
    - Can view assigned customers and their details.
    - Reviews `Policy Requests` raised by customers.
    - Verifies submitted documents (PAN, Address Proof) and approves or rejects requests.
    - Tracks assigned policies.

3. Customer

- **Responsibilities:** End-user of the platform managing their insurance.
- **Actions:**
    - Registers, logs in, and manages their profile.
    - Browses active insurance plans.
    - Raises a `Policy Request` (uploads KYC documents).
    - Once a request is approved by an Agent, the customer can finalize the purchase.
    - Tracks payments, downloads invoices, and files `Claims` in case of incidents.

4. Claims Officer

- **Responsibilities:** Solely deals with the verification and payout of insurance claims.
- **Actions:**
    - Views a dashboard of assigned claims.
    - Reviews claim reasons and uploaded evidence.
    - Approves (provides an approved amount) or Rejects the claim.

---

4. Database Entities

The system uses Entity Framework Core to map C# classes to SQL Server tables.

1. User

Represents all individuals in the system across all roles.

Column Name	Data Type	Description
Id	Guid (PK)	Unique Identifier
FirstName	String (100)	User's First Name
LastName	String (100)	User's Last Name
Email	String (255)	Login Email (Unique)
PasswordHash	String	Bcrypt Hashed Password
Role	UserRole (Enum)	Admin, Agent, Customer, ClaimOfficer
GovernmentId	String	E.g. PAN card number

Address, Phone, DateOfBirth	Various	Profile fields

2. Plan

Represents the available insurance plans.

Column Name	Data Type	Description
Id	Guid (PK)	Unique Identifier
Name	String (200)	Plan Name (e.g., "Silver Auto Shield")
PremiumAmount	Decimal	Monthly or periodic base cost
CoverageAmount	Decimal	Total max payout
DurationInMonths	Int	Length of the plan
PaymentFrequency	String	E.g., Monthly, Yearly

3. PolicyRequest

Temporary entity when a user requests a policy, pending Agent approval.

Column Name	Data Type	Description
Id	Guid (PK)	Unique Identifier
UserId	Guid (FK)	Customer making the request
PlanId	Guid (FK)	Requested plan
AgentId	Guid (FK)	Agent assigned to review
DurationInMonths	Int	Custom duration requested
PanDocumentUrl	String	Uploaded file public URL
AddressProofUrl	String	Uploaded file public URL
Status	Enum	Pending, Approved, Rejected

4. Policy

An approved and active insurance contract.

Column Name	Data Type	Description
Id	Guid (PK)	Unique Identifier
UserId	Guid (FK)	Policy holder
PlanId	Guid (FK)	The insurance plan
AgentId	Guid (FK)	Agent managing the policy

StartDate, EndDate	DateTime	Policy validity period
TotalPremium	Decimal	Sum of all payments to be made
CoverageAmount	Decimal	Max valid claim amount (snapshot)
Status	Enum	Active, Expired, Cancelled

5. PolicyPayment

Tracks individual premium payments made against a policy.

Column Name	Data Type	Description
Id	Guid (PK)	Unique Identifier
PolicyId	Guid (FK)	Associated policy
Amount	Decimal	Payment amount due
DueDate	DateTime	Date by which payment is expected
PaidDate	DateTime	Date of actual payment
Status	Enum	Pending, Paid, Overdue

6. Claim

Represents an incident raised by a customer for financial coverage.

Column Name	Data Type	Description
Id	Guid (PK)	Unique Identifier
PolicyId	Guid (FK)	Affected Policy
UserId	Guid (FK)	Customer claiming
ClaimOfficerId	Guid (FK)	Officer assigned to review
Reason	String	Text description of incident
ClaimAmount	Decimal	Amount customer is requesting
ApprovedAmount	Decimal	Amount approved by officer
Status	Enum	Pending, Approved, Rejected

7. ClaimPayment

Records the actual payout sent to the customer upon Claim Approval.

Column Name	Data Type	Description
Id	Guid (PK)	Unique Identifier

ClaimId	Guid (FK)	Claim this payment belongs to
AmountPaid	Decimal	Equivalent to ApprovedAmount

8. Invoice

A record of generated PDFs stored in the cloud.

Column Name	Data Type	Description
Id	Guid (PK)	Unique Identifier
ReferenceId	Guid	Points to Policy, Claim, or Payment
Type	Enum	PolicyPurchase, PaymentReceipt, ClaimPayout
FileUrl	String	Vercel Blob public link

9. Notification

System alerts sent to users.

Column Name	Data Type	Description
Id	Int (PK)	Auto-increment identifier
UserId	Guid (FK)	Receiver of the notification
Title	String	Short title
Message	String	Detailed content
IsRead	Boolean	Unread/Read status

---

5. DTOs Used in the Application

Data Transfer Objects (DTOs) structure the data flowing between the API and the frontend.

Authentication DTOs

- **`RegisterDto` / `RegisterEmployeeDto`** : Used for creating new accounts. Contains first name, last name, email, password, role, dob, phone.
- **`LoginDto`** : Contains `Email` and `Password` .
- **`AuthResultDto`** : Returned after successful login. Contains `Token` (JWT), `UserId` , `Role` , `FullName` .

Insurance DTOs

- **`CreatePlanDto`** : Used by Admin to add a new `Plan` . Contains fields like Name, Coverage, Premium.
- **`PlanDto`** : Reads Plan details out to the frontend.
- **`PolicyRequestDto`** : Returns policy request info (Customer Name, Status, Document URLs) to Agents and Customers.
- **`PolicyDto`** : Comprehensive view of an active policy, including nested `PlanDto` and `List<PolicyPaymentDto>` .
- **`ClaimDto`** : Used to list claims. Contains ClaimReason, Amounts, Status, OfficerName, DocumentUrl.

- `InvoiceDto` : Lists down accessible invoices for customers `(Id, ReferenceId, Type, FileUrl)` .

User DTOs

- `UserDto` : Exposes basic user info securely (no password hash).
- `UpdateProfileDto` : Used to modify personal user details.
- `UpdateRoleDto` : Used by an admin to change an employee's role.

---

6. Backend Controllers and Endpoints

1. AuthController

- `POST /api/auth/register` : Registers a customer. Returns created user ID.
- `POST /api/auth/login` : Validates credentials. Returns HTTP 200 with JWT token via `AuthResultDto` .

2. UsersController

- `GET /api/users/me` : Returns the currently logged in `UserDto` .
- `PUT /api/users/me` : Updates personal profile.
- `GET /api/users` : (Admin) Retrieves all users in the system.
- `POST /api/users/register` : (Admin) Registers Agents/ClaimOfficers.

3. PlansController

- `GET /api/plans` : Returns all active plans.
- `GET /api/plans/{id}` : Returns details of a specific plan.
- `POST /api/plans` : (Admin) Creates a new plan using `CreatePlanDto` .
- `PUT /api/plans/{id}` : (Admin) Updates existing plan details.

4. PolicyRequestsController

- `POST /api/policy-requests` : (Customer) Creates a request via a `multipart/form-data` payload, uploading PAN and Address documents.
- `GET /api/policy-requests/my` : (Customer) Fetches customer's pending/approved requests.
- `GET /api/policy-requests/assigned` : (Agent) Retrieves requests assigned to this specific agent.
- `POST /api/policy-requests/{id}/approve` : (Agent) Approves the KYC documents.
- `POST /api/policy-requests/{id}/reject` : (Agent) Rejects the request with a typed reason.

5. PoliciesController

- `POST /api/policies/purchase/{requestId}` : (Customer) Converts an *Approved Policy Request* into an Active Policy.
- `GET /api/policies/my` : (Customer) Lists all policies owned by the user.
- `GET /api/policies/assigned` : (Agent) Lists active policies overseen by the agent.
- `POST /api/policies/pay/{paymentId}` : (Customer) Pays an EMI installment.

6. ClaimsController

- `POST /api/claims` : (Customer) Submits a new claim against a policy.
- `GET /api/claims/my` : (Customer) Views past and present claims.
- `GET /api/claims/assigned` : (ClaimOfficer) Views claims pending review.
- `POST /api/claims/{id}/approve` : (ClaimOfficer) Approves the claim and simulates a payout.
- `POST /api/claims/{id}/reject` : (ClaimOfficer) Rejects the claim.

Example JSON Payloads

Endpoint: `POST /api/auth/login` **Purpose:** Authenticate user and receive JWT.

Request Example:

```
{
  "email": "janedoe@example.com",
  "password": "Password123!"
}
```

Response Example:

```
{
  "userId": "c8a45e21-d57b-4026-9f37-1e5b47a11ab4",
  "fullName": "Jane Doe",
  "email": "janedoe@example.com",
  "role": "Customer",
  "token": "eyJhbGciOiJIUzI1NiIsInR5c... (JWT String)"
}
```

Endpoint: `POST /api/policy-requests` **Purpose:** Create a new policy request (Requires `multipart/form-data`).

---

7. Backend Services and Functions

The Application layer abstracts logic through Services handling heavy business rules.

- **`AuthService.cs`**
  - `RegisterAsync(registerDto)` : Hashes passwords with BCrypt, initializes the User entity, returns user data.
  - `LoginAsync(loginDto)` : Validates password. Calls `JwtService` to build a signed token with assigned roles.
- **`PolicyRequestService.cs`**
  - `CreatePolicyRequestAsync(...)` : Accepts file byte arrays. Automatically pushes files to Vercel via `VercelBlobService` , receives URLs, assigns a random Agent, saves the request to DB.
  - `ApproveRequestAsync(id, agentId)` : Updates request status to `Approved` . Identifies agent logic checks.
- **`PolicyService.cs`**
  - `PurchasePolicyAsync(userId, requestId)` : A critical transaction. Takes an Approved request. Snapshots the `Plan` coverage/premium amounts into the new `Policy` . Calculates scheduled `PolicyPayments` (EMIs) based on duration and payment frequency. Triggers `GeneratePolicyInvoice` , saves the PDF via Blob, and creates an `Invoice` database entity.
- **`ClaimService.cs`**
  - `ApproveClaimAsync(claimId, approvedAmount, officerId)` : Processes claim validation, updates `ClaimStatus` to Approved, creates a `ClaimPayment` record representing actual cash transfer, and utilizes the `InvoiceGeneratorService` to render a payout receipt.
- **`InvoiceGeneratorService.cs`**
  - Leverages QuestPDF functionality server-side. Implements definitions for:
    - `GeneratePolicyInvoice(policy, customer)`

        - `GenerateClaimInvoice(policy, claim, customer)`
        - `GeneratePaymentInvoice(policy, payment, customer)`
    - Uses Fluent APIs to draw aesthetic tables, headers, and format logic dynamically.
- **`VercelBlobService.cs`**
    - `UploadFileAsync(fileBytes, fileName, folderName)` : Shoots HTTP `PUT` requests to Vercel Storage APIs with a static generic Bearer Token. Returns a secure public URL string for the file uploaded.

---

8. Frontend Structure

The Angular 19 frontend relies on a clean, scalable folder hierarchy:

```
frontend/src/app/
│
├── components/          # Organizes UI components functionally and by Role
│   ├── admin/           # Dashboard, User/Plan Management
│   ├── agent/           # Policy Requests, Customers list
│   ├── claims-officer/  # Dashboard for active claim queues
│   ├── customer/        # Storefront for plans, My Policies, Submit Claim
│   └── common/          # Login, Register, Profile, unified layout wrappers
│
├── guards/              # authGuard, roleGuard, guestGuard
├── interceptors/        # auth.interceptor process JWT dynamically into Request Headers
├── models/              # TypeScript interface schemas identical to Backend DTOs
├── services/            # Http wrappers interacting with APIs (Auth, Policy, Claim)
└── utils/               # Constants and helper logic
```

- **Modules:** The app uses Angular **Standalone Components**. Features are imported independently per file removing the monolithic `app.module.ts` .
- **Services:** Centralized logic (e.g. `PolicyService` encapsulates `this.http.post('...', body)`).
- **Routing:** Configured centrally in `app.routes.ts` . Lazy loading via `loadComponent: () => import(...)` is implemented throughout for extremely fast load optimizations.
- **Guards (`roleGuard`):** Reads the required role from route setup `data: { role: 'Admin' }` , decodes the local JWT, and determines access securely avoiding frontend tampering.

---

9. Frontend Functions and Logic

1. Login Logic (`components/common/login`)

- Receives user input strings (Email, Password).
- Fires `AuthService.login(payload)` .
- Upon success, the service caches the JWT in `localStorage` .
- Parses the JWT payload payload natively (`jwtDecode` or similar) to read the Role string.
- Injects `Router` to execute programmed navigation: Admin -> `/admin/dashboard` , Customer -> `/customer/dashboard` .

2. Policy Request Integration (`components/customer/customer-plans`)

- Users overview cards mapped from `PlanService.getAll()` .

- A dialog modal captures "Duration", "PaymentFrequency", and utilizes `<input type="file" (change)="onFileSelected($event)">` .
- In `PolicyRequestService.createRequest()` , builds a `FormData` object since standard JSON does not support pure binary streams.
- Relays POST to API logic gracefully providing Toast notifications on success.

3. Agent Policy Review (`components/agent/agent-policy-requests`)

- `ngOnInit` component hook fetches `assigned` tasks via `PolicyRequestService.getAssignedRequests()` .
- The HTML renders hyperlinks bound to `panDocumentUrl` and `addressProofUrl` letting the Agent view the Vercel blobs seamlessly.
- Triggers `approveRequest(id)` triggering the backend API. Re-fetches the observable list to disappear approved tasks intelligently.

---

10. Policy Request and Policy Buying Flow

This sequential process serves as the core engine of the system guaranteeing data verification.

1. **Customer Request:** Customer selects a previously created active Plan. They detail their parameters (PaymentFrequency, DurationInMonths) and upload their PAN and Address Proofs.
2. **System Setup:** A `PolicyRequest` is stored (`Status = Pending`). Uploaded files go instantly to Vercel. The system automatically assigns the request to the **least busy Agent** (the Agent with the fewest pending requests) to ensure an even workload.
3. **Agent Review Phase:** The assigned Agent logs in to their dashboard. They navigate to requests and verify the KYC file evidence.
4. **Agent Approval:** The Agent assesses lower risk and clicks 'Approve'. Request Status transitions to `Approved` . The Agent's job clears.
5. **Purchase Authorization:** The Customer refreshes the interface. Seeing `Approved` , a new 'Purchase' action is unblocked in their view.
6. **Finalizing Policy:** Tapping Purchase fires `PurchasePolicyAsync()` . The backend formally clones plan values, officially activating the policy. It schedules a strict `PolicyPayment` mapping.
7. **Document Generation:** The .NET Server implicitly blocks, initializes QuestPDF, calculates exact financial spreads, generates an aesthetic "Policy Invoice", pushes it again to Blob storage, links via the `Invoice` database table.
8. **Customer Acquisition:** The customer now owns an `Active` Policy, viewing it confidently in their portfolio alongside their PDF receipt.

---

11. Claim Processing Flow

When disasters or casualties occur impacting the insured:

1. **Lodging File:** Customer hits "Submit Claim" against a chosen Active Policy. Providing an explicit typed reason, desired claim amount, and securely attaching photos/evidence.
2. **Claim Staging:** System intercepts generating `Claim` entity under `Pending` status. The system assigns the claim to the **least busy Claims Officer** (based on their current pending claims queue) for efficient processing.
3. **Internal Assessment:** The designated Claims Officer accesses their segregated portal. They have transparent visibility into the customer's policy history, premium payment delays, and the precise Claim Request details.
4. **Conclusion Selection:**

- **Approval Action:** Officer evaluates favorably and inputs the manual `ApprovedAmount`. The system commits the change, immediately triggering QuestPDF to generate a dynamic payout configuration invoice, marks `Status = Approved`, structures a literal `ClaimPayment` log modeling actual banking operations.
- **Rejection Action:** Claim effectively moves to `Rejected` terminating process logic without document generation.

5. **Customer Acknowledgement:** The backend `NotificationService` fires asynchronous records causing the customer to visualize their claim decision dynamically via a top bar component alert.

---

12. Third Party Integrations

1. QuestPDF

- **Purpose:** Utilized for rapid, code-first server-side generation of beautiful PDF invoices dynamically scaling to content without relying on slower HTML-to-PDF middleware.
- **Workflow / Where:** Embedded in `InvoiceGeneratorService`. Invoked explicitly during Policy Activation, Premium Handling, and Claim Conclusion. Logic relies on Fluent API `Container.Page(...)` constructing highly customized header tables, branding, and snapshot calculations into raw `byte[]` streams.

2. Vercel Blob

- **Purpose:** Acts as immutable cloud storage to harbor Images, Proofs, and generated PDFs efficiently off-disk. Keeps backend highly cohesive and drastically scalable.
- **Workflow / Where:** Handled natively in `VercelBlobService` interface. APIs consume incoming `IFormFile`, serialize properties mapped by `HttpClient` to an external HTTP `PUT` relying on an internal static Bearer configuration. Vercel returns JSON parsed directly referencing the new public URL string.

---

13. Security and Authentication

- **JWT Integration (Stateless):** The `.NET` API formulates a temporary JSON Web Token verifying identities immediately eliminating stateful server tracking. Angular's customized `HttpInterceptor` ensures every outgoing HTTP call injects the header `Authorization: Bearer <token>`.
- **Role-Based Authorization Constraints:** The system natively prohibits malicious behavior by guarding functions globally. Endpoints reflect attributes: `[Authorize(Roles = "ClaimOfficer")]`. `AppDbContext` modifications verify this token role implicitly against the logged procedure checking.
- **Password Salting / Hashing:** Implemented fundamentally via the BCrypt hashing algorithm (`BCrypt.Net`), ensuring active internal user structures retain absolute zero access to actual password text, mitigating database breach vectors identically.
- **Data Encapsulation:** Critical API pulls verify contextual IDs dynamically directly from the decoded JWT e.g. `User.FindFirst(ClaimTypes.NameIdentifier)` bypassing query string manipulation risks such as IDOR (Insecure Direct Object Reference) seamlessly.

---

14. Features Implemented in the System

- **Universal User Management:** Complete authentication suites, explicit user profiling, and robust Administrative CRUD control mapping exact Employee assignments.
- **Insurance Plan Cataloging:** Fluid creation, edition, and archiving capability mapped out globally for structured public insurance availability.
- **Asymmetric Underwriting Workflows:** Granular explicit multi-step verification pipeline requiring verifiable binary document uploads reviewed safely by explicitly authorized Agents.

- **Real-Time Financial Logic Management:** Rigid backend EMIs calculation, processing live simulated monetary payments, backed confidently by aesthetic PDF validations automatically filed efficiently.
- **Full lifecycle Claims Navigation:** End-to-end incident reporting mechanics, distinct separate reviews safely walled under designated Claims Officers with strict monetary protections.
- **Cloud Binary Synchronization:** Direct structural deployment configurations ensuring files seamlessly interface using external Serverless Blob Storage frameworks securely generating immutable data references.
- **Event-Driven App Notifications:** Structural, dynamic notification tables generated per-user allowing instantaneous visibility on operational state transformations (Approvals, Rejections, Invoicing).

---

15. Summary

The **Casualty and Disaster Insurance Management System** acts as a remarkably powerful conduit, synchronizing business actions flawlessly between four unique core domains: Administration, Client Servicing (Agents), Dedicated Adjusting (Claims Processing), and the End User Platform.

By architecting logic atop **.NET 8 Clean Architecture**, it mandates absolute code scalability, predictable service actions, explicit domain boundaries, and advanced feature integrations comprising programmatic dynamic data compilation (PDFs) alongside robust file-system proxying (Vercel). The accompanying componentized **Angular 19** interface enforces rapid access, securely limiting visibility per explicitly authenticated identity tokens efficiently. Ultimately, this system resolves the legacy challenges inherent to paperwork-reliant systems securely, reliably mapping an incredibly robust enterprise-grade solution engineered tightly towards modern reliability demands.